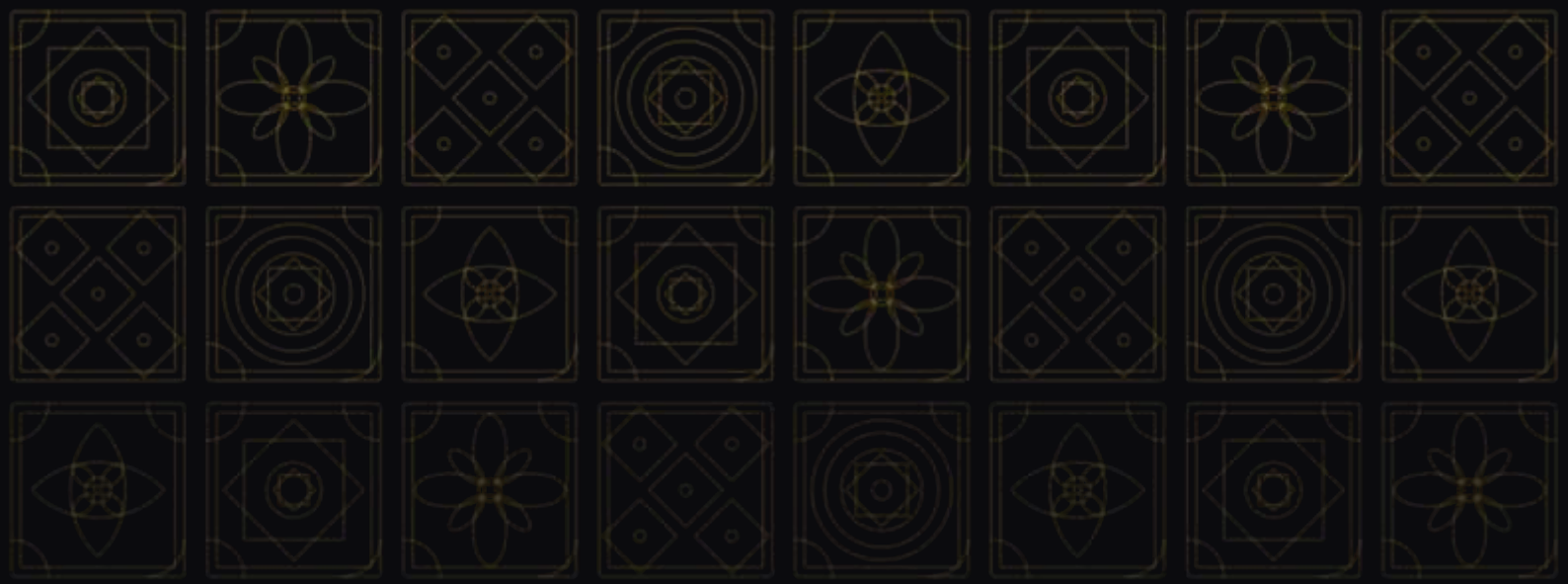




TRAINING PROGRAMME

DevOps Engineering

Build, deliver and run software through one automated pipeline

81 HOURS

FOR Developers, system administrators and IT engineers moving into DevOps roles

PREPARES FOR HashiCorp Terraform Associate (004), core topics · GitHub Actions (GH-200), core topics · LPI DevOps Tools Engineer 2.0 (701-200), core topics



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This programme follows a single change from a developer's commit to a monitored application running on AWS. Participants set up Git collaboration, build continuous integration and delivery pipelines with Jenkins and GitHub Actions, add automated tests, quality gates and versioned artifacts, provision the infrastructure with Terraform and configure servers with Ansible, then watch the result through metrics, logs and alerts. The same application is carried through every phase, almost every chapter ends with a lab, and the programme closes with a final project in which teams assemble the whole delivery chain and present it.

At a glance

#	PHASE	FOCUS	HOURS
01	DevOps Culture & Collaboration	DevOps principles, Git and team workflows	9 h
02	Continuous Integration & Delivery	Jenkins, automated tests, SonarQube, Nexus, deployment strategies, GitHub Actions	24 h
03	Infrastructure as Code	Terraform provisioning on AWS, Ansible configuration management	24 h
04	Observability	Prometheus metrics and alerts, Grafana dashboards and Loki logs	6 h
05	Final Project	End-to-end delivery chain, presented to the group	9 h
	Buffer	Catch-up, revision, extra lab time	9 h
	Total		81 h

Key facts

AUDIENCE	Developers, system administrators and IT engineers moving into DevOps roles
PREREQUISITES	Linux command line and Bash (course: Linux & Bash Scripting); Docker and Kubernetes basics (course: Docker & Kubernetes). This course builds on Docker & Kubernetes and does not repeat it.
DURATION	81 h: 72 h of chapters + 9 h buffer
FORMAT	Short lectures followed by guided labs; one application carried through every phase
LAB ENVIRONMENT	One laptop per participant (16 GB RAM minimum, Docker, VS Code); phases 1 and 2 run locally, phase 3 moves the stack to AWS. One AWS sandbox account per team, provided by the organiser; resources destroyed at the end of each lab to control cost
ASSESSMENT	A lab validated at the end of each phase and a final project presented to the group; certificate of completion for participants who complete both
OUTCOME	Participants can take an application from commit to production through an automated pipeline with tests, quality gates, versioned artifacts, infrastructure as code, and monitoring with alerts

PHASE 01

9 H

DevOps Culture & Collaboration

DevOps starts with how a team works, then with how it shares code. This phase sets the vocabulary and metrics used for the rest of the programme and gives every participant a working command of Git and pull-request based collaboration. The phase ends with a validated lab: a team repository with protected branches, a reviewed pull request and a tagged release.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	DevOps Culture	CALMS, DORA software delivery metrics (throughput and instability, including rework rate), Agile and Kanban, value stream mapping, where automation pays off	Lecture + Workshop	3 h
1.2	Git Fundamentals	How Git stores history (commits, trees, refs), init, clone, add, commit, log, diff, .gitignore, commit message conventions	Lecture + Lab	1.5 h
1.3	Branching & Merging	Branch model, fast-forward and three-way merges, rebase, cherry-pick, conflict resolution	Lecture + Lab	1.5 h
1.4	Remote Repositories & Pull Requests	fetch, pull, push, SSH vs HTTPS authentication, forks, origin and upstream, pull requests and code review	Lecture + Lab	1.5 h
1.5	Team Workflow	GitFlow vs trunk-based development, branch protection rules, release tags, semantic versioning	Lecture + Lab	1.5 h

PHASE 02

24 H

Continuous Integration & Delivery

Participants build the same pipeline twice: on Jenkins, a widely used self-hosted CI server, and on GitHub Actions. Tests, quality gates and an artifact repository make sure that only tested, versioned builds are deployed, and the phase ends with the release strategies that keep a deployment reversible. Validated lab: the course application built, tested, analysed, published to Nexus and deployed to the local cluster by both pipelines.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Jenkins Foundations	Installation, plugins, security settings, controller and agents, job types	Lecture + Lab	1.5 h
2.2	Triggers & Credentials	Webhooks, SCM polling, scheduled builds, credential types and their safe use	Lecture + Lab	1.5 h
2.3	Jenkins Pipelines	Declarative Jenkinsfile, stages and steps, post actions, parameters, environment variables	Lecture + Lab	3 h
2.4	Scaling Jenkins	Distributed agents, parallel stages, shared libraries for reusable pipeline code	Lecture + Lab	1.5 h
2.5	Automated Testing in the Pipeline	Unit and integration test stages, JUnit reports, failing fast, disposable test environments with containers	Lecture + Lab	1.5 h
2.6	Code Quality with SonarQube	Static analysis, coverage import, quality profiles, quality gates that stop the pipeline	Lecture + Lab	3 h
2.7	Artifact Management with Nexus	Hosted, proxy and group repositories, Nexus as a Docker registry, versioning, cleanup policies, publishing from the pipeline	Lecture + Lab	3 h
2.8	Deploying from Jenkins	Build, tag and push images to Nexus, deploy to Kubernetes with Helm, promotion between environments	Lab	1.5 h
2.9	Deployment Strategies & Rollback	Rolling, blue/green and canary releases, rollback, feature flags; push-based deployment vs GitOps (Argo CD, taught in Docker & Kubernetes)	Lecture + Lab	1.5 h
2.10	GitHub Actions	Workflows, events, jobs and steps, expressions and contexts, hosted and self-hosted runners, matrix builds, caching, artifacts	Lecture + Lab	3 h
2.11	Secure Delivery & Reuse with Actions	Secrets, environments and required reviewers, OIDC federation to AWS without long-lived keys, reusable workflows, custom composite, JavaScript and Docker actions, organisation policies and runner groups	Lecture + Lab	3 h

PHASE 03

24 H

Infrastructure as Code

Infrastructure is described in code, reviewed like code and applied by the pipeline. Terraform creates the AWS environment for the course application; Ansible then configures the servers that run the team's tooling. Each tool ends with a validated project that the final project reuses.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	Terraform Foundations	Providers, HCL, resources, data sources, dependency graph, init, plan and apply, fmt and validate	Lecture + Lab	3 h
3.2	State & Variables	Variables and validation, outputs, state and the terraform state commands, S3 remote backend and locking, HCP Terraform	Lecture + Lab	3 h
3.3	Modules & Environments	Writing and versioning modules, the Terraform Registry, workspaces, lifecycle rules, import and moved blocks, code organisation	Lecture + Lab	3 h
3.4	Terraform Project	Provision the course environment on AWS: VPC, EKS cluster, RDS database and EC2 hosts for the tooling	Lab	3 h
3.5	Ansible Basics	Declarative vs imperative, configuration drift, architecture, static and AWS dynamic inventories, ad hoc commands	Lecture + Lab	3 h
3.6	Playbooks	YAML, plays and tasks, core modules, handlers, conditionals, loops, idempotency	Lecture + Lab	3 h
3.7	Variables, Templates & Vault	group_vars and host_vars, facts, Jinja2 templates and filters, Ansible Vault for secrets	Lecture + Lab	3 h
3.8	Roles	Role structure, defaults, dependencies, Ansible Galaxy, designing reusable roles	Lecture + Lab	1.5 h
3.9	Ansible Project	Configure the EC2 tool servers (Jenkins controller and agents, SonarQube, Nexus) with roles, then rerun to prove idempotency	Lab	1.5 h

PHASE 04

6 H

Observability

A delivery chain is only finished when the team can see what runs in production. Participants instrument the course application, build dashboards from metrics and logs, and route alerts to the right people. Validated lab: a dashboard for the course application and an alert that fires and reaches its contact point.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Metrics & Alerting with Prometheus	Metrics model, exporters, instrumenting the application with OpenTelemetry and Prometheus client libraries, PromQL, recording and alerting rules, Alertmanager routing	Lecture + Lab	3 h
4.2	Dashboards & Logs with Grafana	Dashboards, Loki for logs, Grafana alert rules, contact points and notification policies, service level objectives	Lecture + Lab	3 h

PHASE 05

9 H

Final Project

Teams of two or three assemble everything from the programme into one working delivery chain for a provided application, then present it as they would to an engineering manager.

#	CHAPTER	CONTENT	FORMAT	HOURS
5.1	Brief & Design	Read the brief, draw the target architecture, split the work into a backlog, set up the team repository and AWS environment from the phase projects	Project	3 h
5.2	Build	Git workflow; CI with tests, quality gate and versioned artifacts; AWS infrastructure with Terraform; tool servers configured by Ansible; deployment to EKS with a rollout strategy	Project	3 h
5.3	Monitor & Document	Metrics, logs, dashboards and alerts; README, architecture diagram, runbook	Project	1.5 h
5.4	Demo & Review	Live demo of a change going from commit to production, then a rollback; questions and feedback	Project	1.5 h

FINAL PROJECT

From Commit to Production

A change pushed to Git is tested, analysed, versioned and deployed automatically to AWS infrastructure that the team provisioned in code. Its health is visible on dashboards with alerts, and a bad release can be rolled back.

STACK Git, GitHub, Jenkins or GitHub Actions, SonarQube, Nexus, Terraform, Ansible, Docker, Kubernetes (EKS), Helm, Prometheus, Grafana, Loki

DELIVERABLE Repository with pipeline and infrastructure code, architecture diagram, runbook, live demo